TTL and Expiration

how long data may live, and why to add randomness

Phase 3 videos on YouTube

> **In one line:** a TTL is how long a cache entry stays valid (absolute: fixed from the write; sliding: reset on every read); choose the longest TTL your staleness budget allows, invalidate explicitly for data that must change fast, and add jitter so thousands of keys don't expire at the same moment and flood the database.

1 Too long, or too short

You cache product prices for an hour. A sale starts, prices drop, and customers keep seeing the old price for 50 minutes. So you set the TTL to one second, and now the cache barely helps. How long should data live in a cache, and why do smart teams add randomness to that number?

2 Absolute and sliding TTLs

A **TTL**, time to live, is how long a cache entry stays valid. With an **absolute** TTL, the entry expires a fixed time after it was written, no matter how often it's read. With a **sliding** TTL, every read resets the clock, so popular entries stay alive while idle ones expire.

Sliding TTLs suit **sessions** (log out after 30 idle minutes). Absolute TTLs suit data that changes on its own schedule, like prices, because a popular entry with a sliding TTL could live forever, stale.

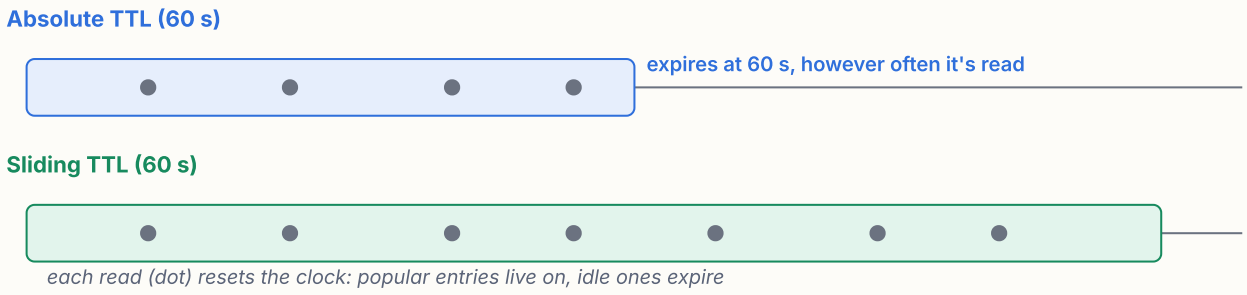


Figure 1. An absolute TTL expires on schedule; a sliding TTL is extended by every read.

3 Choosing the number: the staleness budget

Start from a **staleness budget**: how old can this data be before it hurts? An exchange rate might tolerate a minute; a product description, an hour; a user's permissions, only seconds. Then weigh the hit ratio: longer TTLs mean more hits and less database load. **Pick the longest TTL the staleness budget allows**, and combine it with **invalidation on writes** (lesson 22) for data that must update quickly. Then the TTL is only a safety net for missed invalidations.

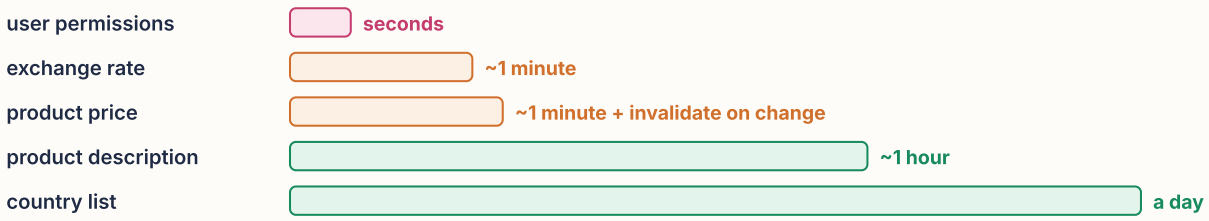


Figure 2. Staleness budgets differ by data: the TTL follows from how much staleness the business can accept.

4 Synchronised expiry, and jitter

Suppose you warm 10,000 keys at startup, all with a 60-second TTL. Sixty seconds later they all expire at the same moment, every request misses at once, and the database gets a sudden wall of queries. Then the keys are refilled together, and it happens again 60 seconds later: a heartbeat of pain.

The fix is **jitter**: add a small random amount to each TTL. Instead of exactly 60 seconds, use `60 ± 10`. Expirations spread across a 20-second window, and the database sees a smooth trickle of misses instead of a spike.

```
ttl = base + random.uniform(−spread, +spread) e.g. 60 + uniform(−10, 10)
```

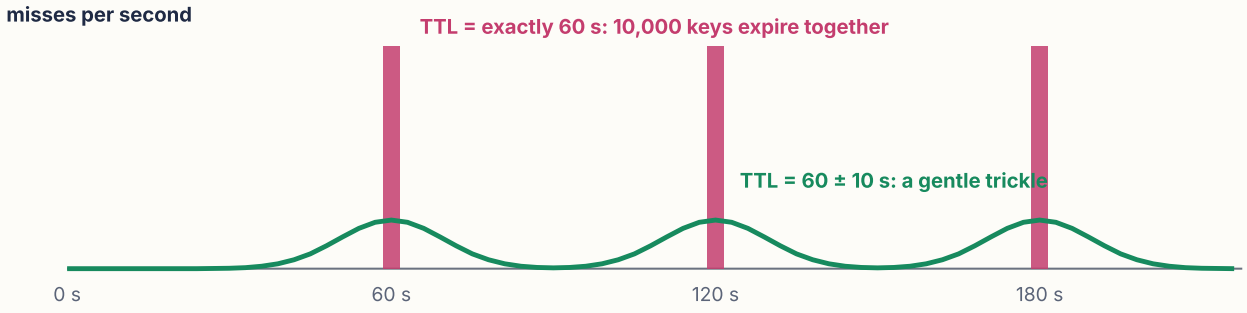


Figure 3. Without jitter, keys written together expire together and produce miss spikes; with jitter the misses spread out.

WORKED EXAMPLE: PRODUCT PRICES

Prices are cached for 60 seconds ± 10 seconds of random jitter. They're never more than about 70 seconds stale, which the business accepts for browsing. A price change for a sale also **deletes the key immediately**, so sale prices appear within milliseconds; the TTL only matters if an invalidation is lost. Because expirations are spread out, database load stays flat. (Checkout still reads the price from the database.)

5 How Redis expires keys

- **Lazy expiry**: when you access a key, Redis checks its TTL and deletes it if it has expired.
- **Active expiry**: a background job runs about 10 times a second, samples random keys that have TTLs (20 per round), and deletes the expired ones; if many of the sample were expired, it repeats straight away.

Together they keep memory tidy without scanning every key. Set TTLs with `EXPIRE key 60` or `SET key value EX 60`, and inspect them with `TTL key` (−1 means no TTL, −2 means the key doesn't exist).

COMMON MISTAKES

- No TTL at all: one missed invalidation leaves wrong data forever.
- The same fixed TTL for keys written in bulk, causing synchronised expiry.
- Sliding TTLs on data that changes, so hot entries never refresh.
- Overwriting a key with `SET` without `EX`, which silently removes its TTL.

6 Key terms

TTL: time to live: how long an entry is valid

Sliding TTL: every read extends the lifetime

Synchronised expiry: many keys expiring at the same moment

Lazy expiry: delete expired keys when accessed

Absolute TTL: expires a fixed time after the write

Staleness budget: how old data may be before it hurts

Jitter: random spread added to TTLs

Active expiry: background sampling of expired keys

7 Interview questions

Q1. How do you choose a TTL? BASIC

Start from the staleness budget, how old the data can be before it causes harm, and pick the longest TTL within it to maximise hits, with explicit invalidation for data that must change quickly.

Q2. Why add jitter to TTLs? BASIC

To spread expirations randomly, so many keys don't expire at the same moment and flood the database with misses.

Q3. Absolute or sliding TTL for a login session? BASIC

Sliding, usually with an absolute maximum: active users stay logged in, idle sessions expire, and every session ends eventually (say 30 minutes idle, 12 hours maximum).

Q4. Every 5 minutes, database CPU spikes for a few seconds. The cache uses a 300 s TTL. What's happening? SCENARIO

Keys were written together (at startup or by a batch) and expire together, so misses arrive in a burst every TTL. Add jitter, and coalesce concurrent misses for the same key.

Q5. How does Redis find expired keys without scanning everything? TRICKY

Lazily on access, plus an active cycle about 10 times a second that samples keys with TTLs and deletes expired ones, repeating while many samples are expired.

Q6. If you invalidate on every write, why still set a TTL? TRICKY

As a safety net: invalidations can be lost (a crash between the database write and the delete, a bug, a direct database fix), and the TTL bounds how long wrong data can live.

Next: SD-F-006 · LRU Cache